

ALU + FIFO

Verilog HDL Project

8-bit ALU integrated with 8-bit synchronous FIFO
Design Documentation & Source Code

Name	Jaimin Doshi
Project	8-bit ALU integrated with 8-bit FIFO
Tools	Quartus II + ModelSim
Document	Edited & Corrected Source Code Document

1. Project Structure

This project is divided into four Verilog files. The ALU performs the selected arithmetic or logic operation, the FIFO stores the resulting values in First-In First-Out order, the top-level module connects the two blocks, and the testbench verifies the complete design in ModelSim.

File	Type	Description
alu.v	Synthesizable	8-bit combinational Arithmetic Logic Unit.
fifo.v	Synthesizable	8-bit wide, 8-entry synchronous FIFO.
alu_fifo_top.v	Synthesizable	Top-level hardware module that connects ALU output to FIFO input.
alu_fifo_tb.v	Simulation only	Testbench that generates clock, reset, test vectors and monitoring output.

2. ALU Module (alu.v)

The 8-bit ALU accepts **A**, **B** and a 3-bit **opcode**. It supports eight operations and generates result, carry and zero status.

Opcode	Operation	Expression
000	Addition	$A + B$
001	Subtraction	$A - B$
010	Bitwise AND	$A \& B$
011	Bitwise OR	$A B$
100	Bitwise XOR	$A \wedge B$
101	Bitwise NOT	$\sim A$
110	Increment	$A + 1$
111	Decrement	$A - 1$

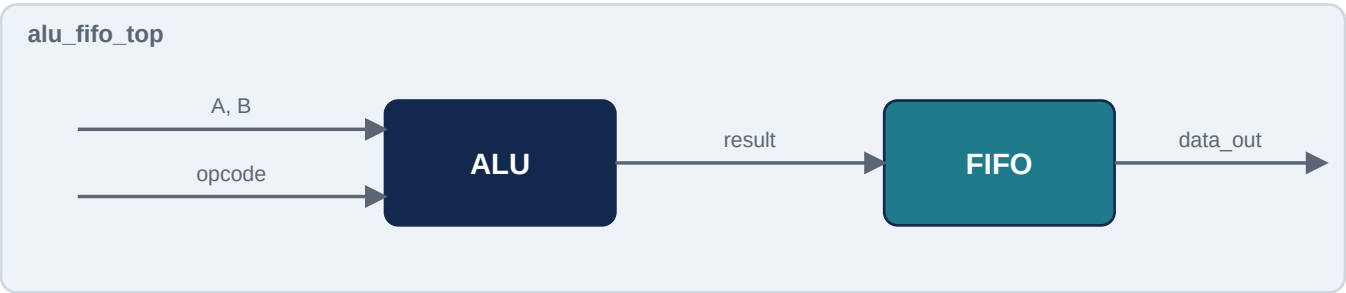
3. FIFO Module (fifo.v)

The synchronous FIFO is 8 bits wide and stores 8 entries. It uses an 8×8 memory array, write pointer, read pointer and a 4-bit count.

- **write_en** stores data when the FIFO is not full.
- **read_en** reads the oldest stored value when the FIFO is not empty.
- **full** is asserted when all eight entries are occupied.
- **empty** is asserted when there are no valid entries.
- **reset** clears pointers, count and output data.

4. Top-Level Integration (alu_fifo_top.v)

The top-level synthesizable module instantiates the ALU and FIFO. The ALU result is routed directly to the FIFO data input and also exposed as **alu_result**. For Quartus synthesis, **alu_fifo_top** is the top-level entity.



5. Testbench (alu_fifo_tb.v)

The testbench is simulation-only. It generates the clock and reset, applies ALU test vectors, writes four ALU results into the FIFO, reads them back and prints the signals using \$monitor. The ModelSim simulation top should be **alu_fifo_tb**; it should not be used as the Quartus synthesis top.

6. Expected Simulation Results

The four ALU results written into the FIFO are expected to be **0F**, **0C**, **88** and **EE**. When the FIFO is read, the same values should appear in the same order. At reset, **empty** should be 1 and **full** should be 0. After all four reads, **empty** should return to 1.

Test	Operation	Expected ALU	FIFO Read
1	0A + 05	0F	0F
2	14 - 08	0C	0C
3	AA & CC	88	88
4	AA CC	EE	EE

7. File Usage & Simulation Workflow

- **Quartus synthesis top:** alu_fifo_top.v
- **ModelSim simulation top:** alu_fifo_tb.v
- **Required source files:** alu.v, fifo.v and alu_fifo_top.v.
- **Compilation order:** alu.v -> fifo.v -> alu_fifo_top.v -> alu_fifo_tb.v.
- **Expected FIFO sequence:** Write 0F, 0C, 88, EE; Read 0F, 0C, 88, EE.

8. Corrections Made

- Standardized module names and instantiation syntax.
- Corrected Verilog formatting and indentation throughout all four source files.
- Corrected the timescale declaration formatting.
- Kept the ALU opcode mapping consistent with the documented project.
- Kept FIFO width, depth, pointer and count definitions consistent.
- Made the testbench sequence and expected results consistent with the documented simulation.

Source Code

alu.v

8-bit combinational ALU with carry and zero flags.

```
`timescale 1ns/1ps

module alu (
    input    [7:0] A,
    input    [7:0] B,
    input    [2:0] opcode,
    output reg [7:0] result,
    output reg      carry,
    output          zero
);

always @(*) begin
    result = 8'b0;
    carry  = 1'b0;

    case (opcode)
        3'b000: begin
            {carry, result} = A + B;
        end
        3'b001: begin
            result = A - B;
            carry  = (A < B);
        end
        3'b010: begin
            result = A & B;
        end
        3'b011: begin
            result = A | B;
        end
        3'b100: begin
            result = A ^ B;
        end
        3'b101: begin
            result = ~A;
        end
        3'b110: begin
            result = A + 1'b1;
        end
        3'b111: begin
            result = A - 1'b1;
        end
        default: begin
            result = 8'b0;
        end
    endcase
end

assign zero = (result == 8'b0);

endmodule
```

fifo.v

8-entry synchronous FIFO with full/empty status.

```
`timescale 1ns/1ps

module fifo (
    input          clk,
    input          reset,
    input [7:0]    data_in,
    input          write_en,
    input          read_en,
    output reg [7:0] data_out,
    output         full,
    output         empty
);

reg [7:0] mem [0:7];
reg [2:0] wr_ptr;
reg [2:0] rd_ptr;
reg [3:0] count;

assign full  = (count == 4'd8);
assign empty = (count == 4'd0);

always @(posedge clk) begin
    if (reset) begin
        wr_ptr  <= 3'd0;
        rd_ptr  <= 3'd0;
        count   <= 4'd0;
        data_out <= 8'd0;
    end
    else begin
        if (write_en && !full) begin
            mem[wr_ptr] <= data_in;
            wr_ptr      <= wr_ptr + 1'b1;
        end

        if (read_en && !empty) begin
            data_out <= mem[rd_ptr];
            rd_ptr   <= rd_ptr + 1'b1;
        end

        case ({(write_en && !full), (read_en && !empty)})
            2'b10: count <= count + 1'b1;
            2'b01: count <= count - 1'b1;
            default: count <= count;
        endcase
    end
end

endmodule
```

alu_fifo_top.v

Top-level synthesizable module (Quartus top-level entity).

```
`timescale 1ns/1ps

module alu_fifo_top (
    input      clk,
    input      reset,
    input  [7:0] A,
    input  [7:0] B,
    input  [2:0] opcode,
    input      write_en,
    input      read_en,
    output [7:0] alu_result,
    output [7:0] data_out,
    output      carry,
    output      zero,
    output      full,
    output      empty
);

wire [7:0] result_wire;

assign alu_result = result_wire;

alu ALU_UNIT (
    .A      (A),
    .B      (B),
    .opcode (opcode),
    .result (result_wire),
    .carry  (carry),
    .zero   (zero)
);

fifo FIFO_UNIT (
    .clk      (clk),
    .reset     (reset),
    .data_in  (result_wire),
    .write_en (write_en),
    .read_en  (read_en),
    .data_out (data_out),
    .full     (full),
    .empty    (empty)
);

endmodule
```

alu_fifo_tb.v

Simulation-only testbench (ModelSim top).

```

`timescale 1ns/1ps

module alu_fifo_tb;

    reg        clk;
    reg        reset;
    reg [7:0] A;
    reg [7:0] B;
    reg [2:0] opcode;
    reg        write_en;
    reg        read_en;

    wire [7:0] alu_result;
    wire [7:0] data_out;
    wire        carry;
    wire        zero;
    wire        full;
    wire        empty;

    alu_fifo_top DUT (
        .clk        (clk),
        .reset       (reset),
        .A           (A),
        .B           (B),
        .opcode      (opcode),
        .write_en    (write_en),
        .read_en     (read_en),
        .alu_result  (alu_result),
        .data_out    (data_out),
        .carry       (carry),
        .zero        (zero),
        .full        (full),
        .empty       (empty)
    );

    always #5 clk = ~clk;

    initial begin
        clk      = 0;
        reset    = 1;
        A        = 8'h00;
        B        = 8'h00;
        opcode   = 3'b000;
        write_en = 0;
        read_en  = 0;

        #10;
        reset = 0;

        // Test 1: ADD - 10 + 5 = 15 (0F)
        A = 8'd10;
        B = 8'd5;
        opcode = 3'b000;
        #2;
        write_en = 1;
        #10;
        write_en = 0;

        // Test 2: SUB - 20 - 8 = 12 (0C)
        A = 8'd20;
        B = 8'd8;
        opcode = 3'b001;

```

```

#2;
write_en = 1;
#10;
write_en = 0;

// Test 3: AND - AA & CC = 88
A = 8'hAA;
B = 8'hCC;
opcode = 3'b010;
#2;
write_en = 1;
#10;
write_en = 0;

// Test 4: OR - AA | CC = EE
A = 8'hAA;
B = 8'hCC;
opcode = 3'b011;
#2;
write_en = 1;
#10;
write_en = 0;

// Read four FIFO entries
#10;
read_en = 1;
#10;
read_en = 0;
#10;
read_en = 1;
#10;
read_en = 0;
#10;
read_en = 1;
#10;
read_en = 0;
#10;
read_en = 1;
#10;
read_en = 0;

#20;
$finish;
end

initial begin
    $monitor(
        "Time=%0t | A=%h | B=%h | Opcode=%b | ALU=%h | Write=%b | Read=%b | FIFO_Out=%h | Full=%b | Empty=%b",
        $time, A, B, opcode, alu_result,
        write_en, read_en, data_out, full, empty
    );
end

endmodule

```